hello-ml-research-group

A repo for the "hello world" exercise with Github for the research group. This repo is intended to be used as a space to get comfortable with using Github, and this readme is a tutorial on basic github use, including best practices. This is intended to be followed using the command line or the VS Code GUI, options for both given for all steps. ***

Prerequisites

- Download VS Code
- Download Git
- Create a GitHub account
- Membership in the Stellenbosch-University-Process-Eng organization.
- Log into your Github account on VSCode, on the left hand side on the bottom of the screen of VSCode, person looking icon.
- Configure your git identity in the command line / terminal (Powershell on Windows or Linux on Mac): `bash  git config --global user.name "Your Name"  git config --global user.email "your.email@example.com"` You can do this in VSCode itself by opening a terminal using `Ctrl + Shift + `

Find your organization and locate a repository

1. Sign in to Github.com in your browser. Click your profile and switch to the Stellenbosch-University-Process-Eng organization.
2. Open the hello-ml-research-group repository.
3. Open the repository's main page and locate the green Code dropdown, this is where you'll copy the repository URL for cloning.

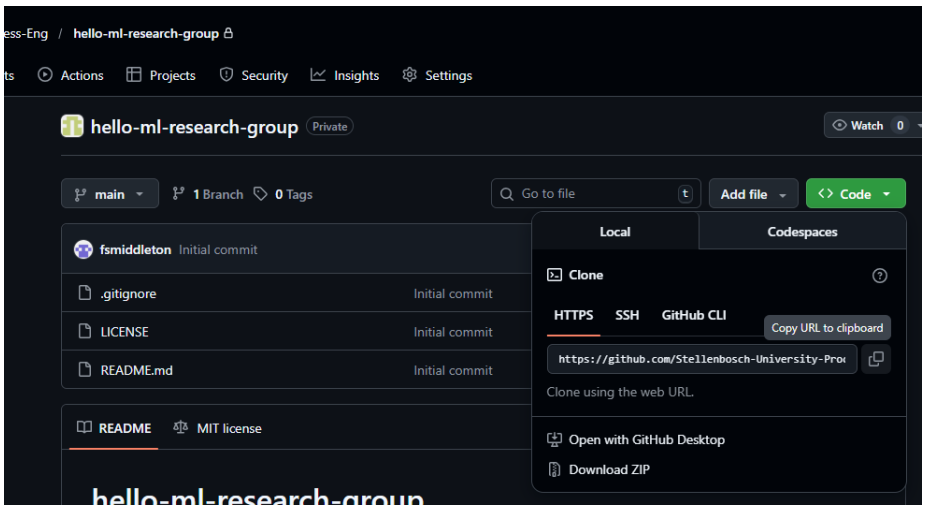


Figure 1: GitHub organization view with a repository selected

Clone the repository via HTTPS

Terminal

```
# Choose a parent directory eg.
cd "<location you want to clone to>"
# Clone the repo in that directory
git clone https://github.com/<ORG>/<REPO>.git
# navigate to the repo
cd <REPO>
# open the repo in VS code
code .
```

The commands in the terminal

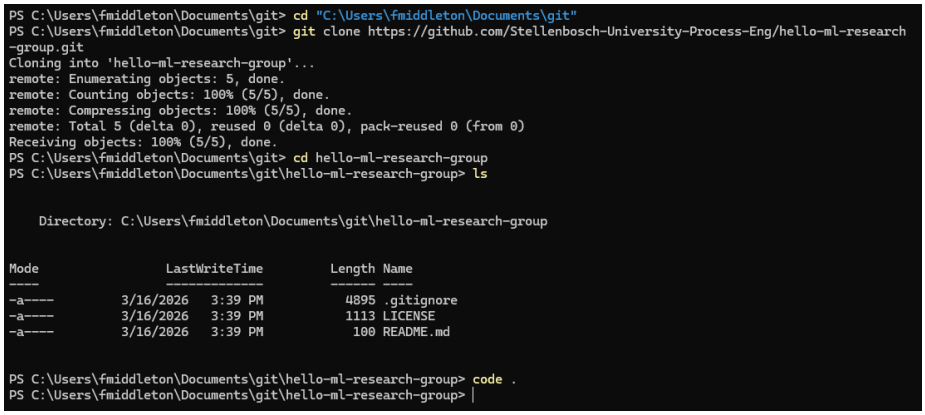


Figure 2: Command line cloning a repo

VS code opening from the terminal command `code .`

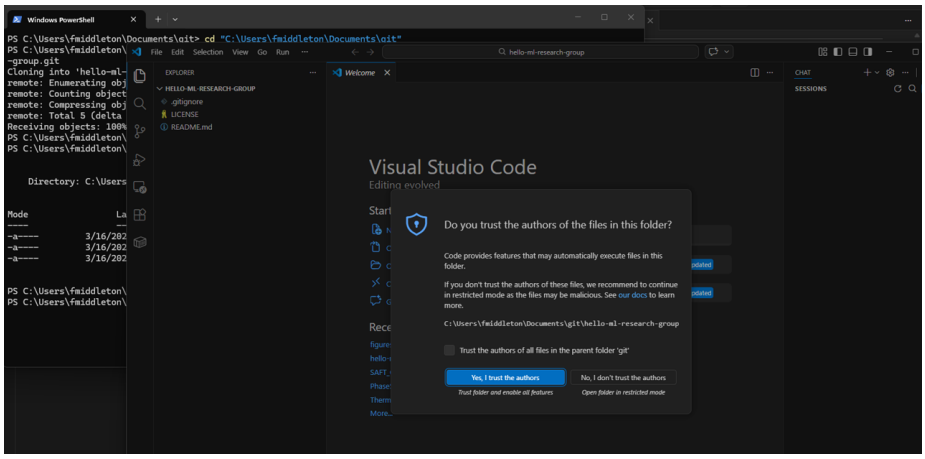


Figure 3: Open VS code from terminal

VS Code GUI

- Open VS Code → Source Control view → Clone Repository → paste the HTTPS URL → choose a local folder → Open when prompted.

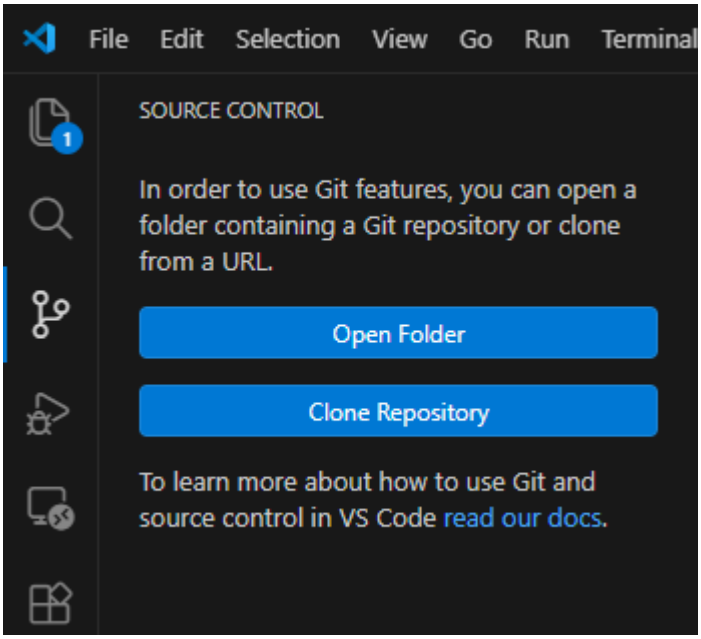


Figure 4: VS Code "Clone Repository"

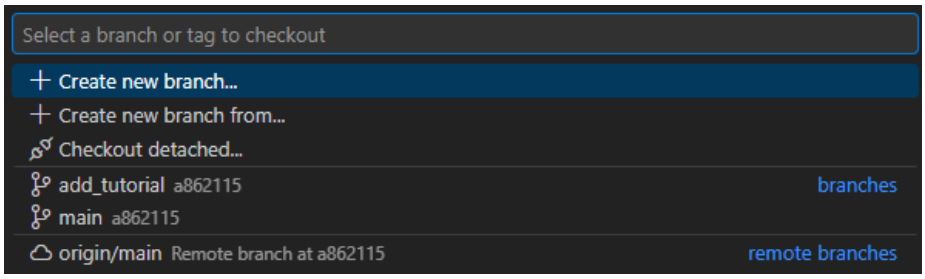
VS Code will detect Git and offer the full Source Control workflow.

Create a topic branch for your change

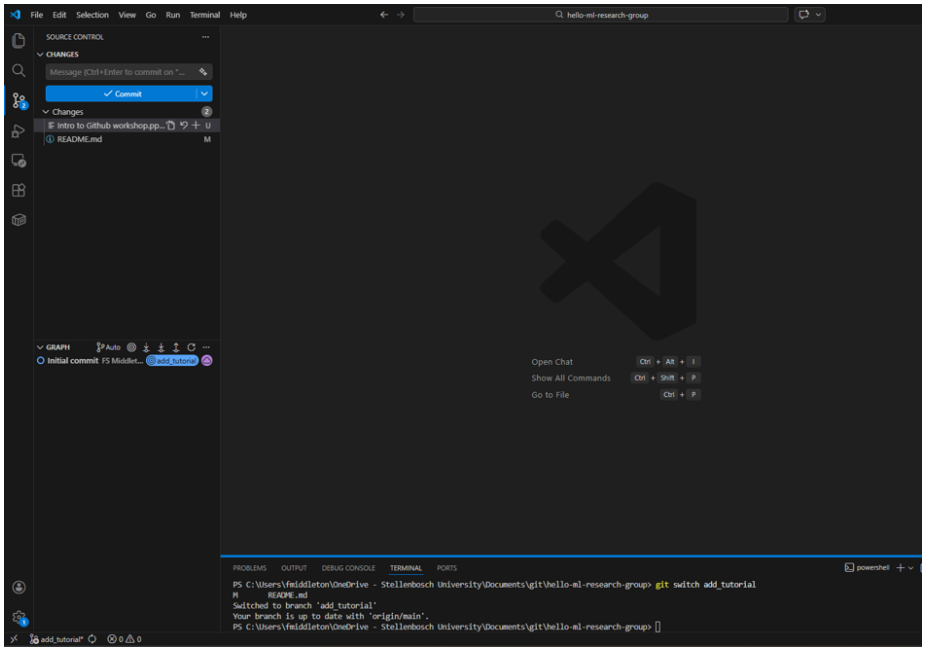
- Branches are places where we will make code changes that our main branch can "pull" in with a "Pull Request". ### Terminal

```
git switch -c <my_branch_name>
```

- `git switch -c` both creates (-c) and checks out (switch) the branch. The newer `switch` command focuses solely on branch operations, reducing the ambiguity of the older, multi-purpose `checkout`.
- What is the difference between git switch and checkout? ### VSCode GUI Use the branch name in the Status Bar → Create New Branch → provide `<my_branch_name>`. The editor will switch you to the new branch.



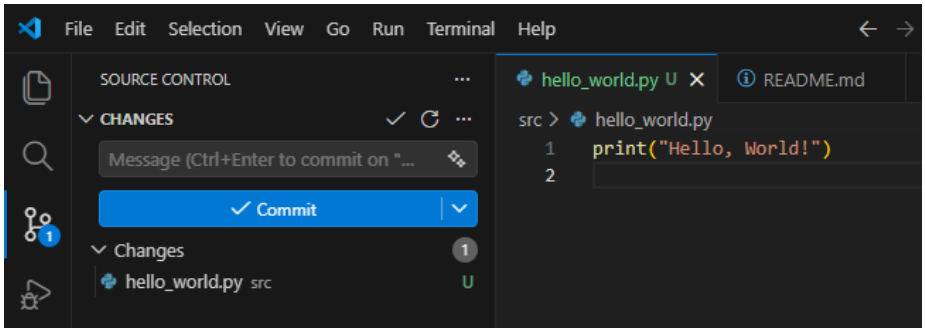
After using the terminal or GUI, your window will look something like this:



***

Make a minimal feature change

Create a file called "hello_world.py" and place the code `print("Hello, World!")` in the file. Save the file. It should now appear on your source control list.



***

Stage and commit your changes

- A commit records a snapshot of the staged content and links it into the repository's history DAG.
- You **cannot** leave out the **commit message**, or the commit will not work.
- Commit message guidance: Use an imperative, concise subject and (when needed) a body explaining intent. Chris Beams' "Seven Rules" are widely cited for clarity and consistency.

Terminal

Stage and commit the feature change, or all changes, and push changes to the remote branch.

```
# Stage the change (specific file)
git add src/hellow_world.py
# Stage all changes
git add .
# Commit the change
git commit -m "feat: add hello world file"
# Push your commit to remote
git push -u origin <my_branch_name>
```

VSCode GUI

VS Code GUI mirrors the command-line operations. * Stage specific files in Source Control, using the "+" icons. Or stage all of them using the "+" icon next to "Changes" * Type your commit message, and press Commit. * Refresh the branch by allowing all pulls and pushes to happen that have been fetched from the remote. This should only be used if you know you will have no merge conflicts.

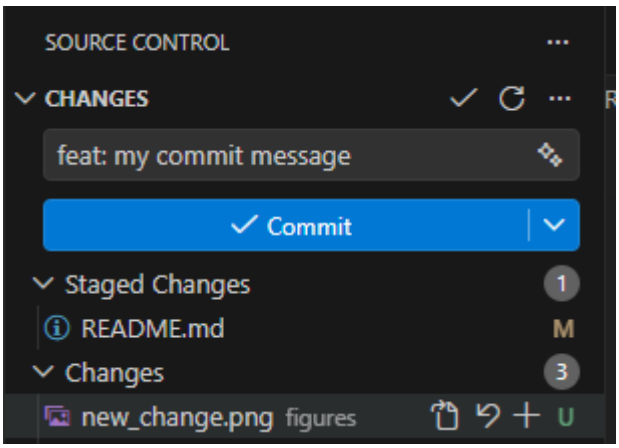


Figure 5: GUI staging and committing

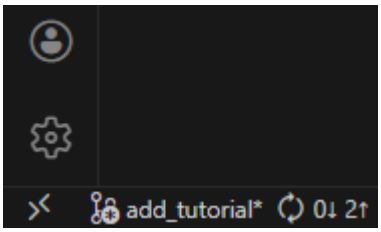


Figure 6: Push pull indicator

Open a Pull Request on GitHub.com

A pull request is a proposal to merge changes from your branch into a target branch (usually `main`). It provides a structured space for review, discussion, checks, and, eventually, integration.

1. Navigate to your repository on GitHub. You will typically see a prompt: "Compare & pull request" for your newly pushed branch.
2. Click it and fill out:
   - Title: short imperative summary, e.g., "Add `hello_world`".
   - Description: context, rationale, and any risks.
3. Submit the PR.

See below what the GUI looks like when you have a branch to open a PR for, and what the PR editing window looks like.

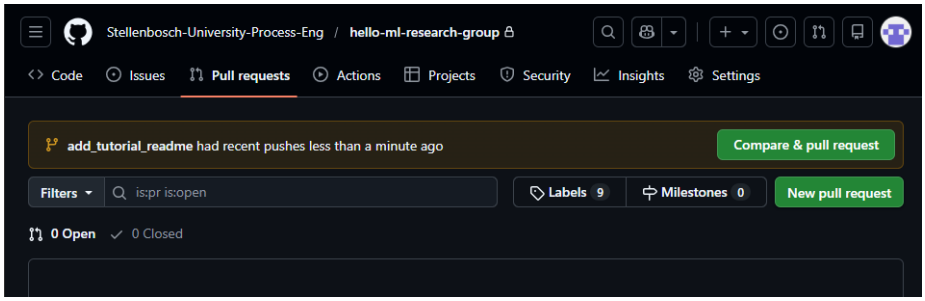


Figure 7: Open a PR

This is what the PR looks like upon opening, before adding any reviewers and for a repo without branch protection.

Please note that you can still make changes to a branch after a PR has been opened.

Assign a colleague, get a review, and merge after approval

- In the PR sidebar, request reviewers and/or assign the PR to a specific colleague.

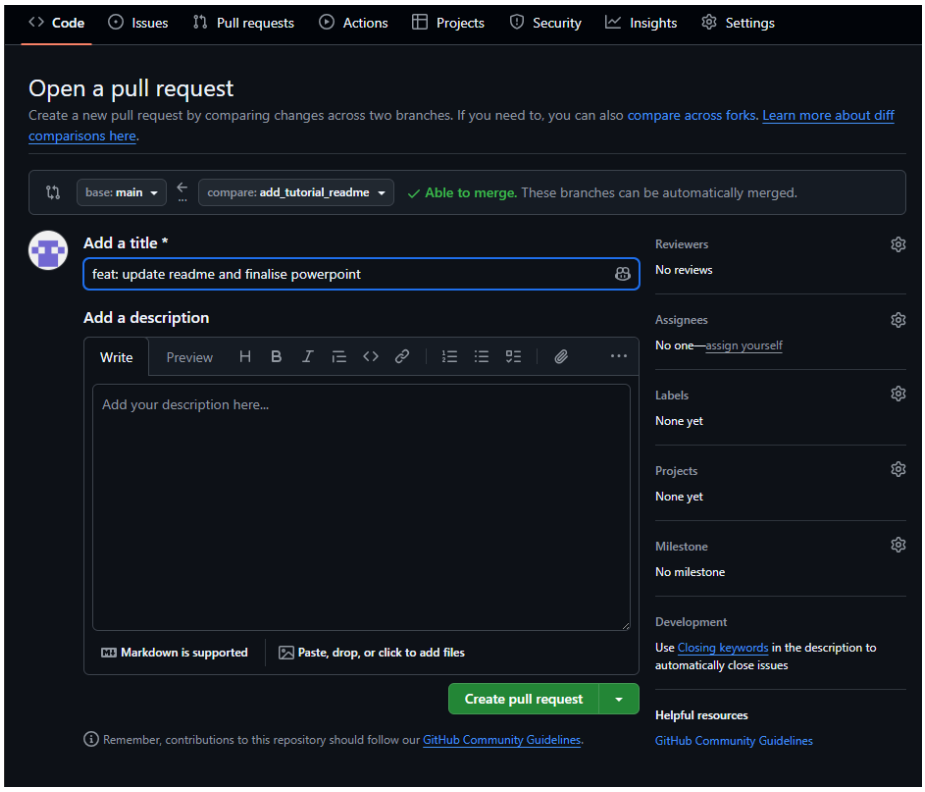


Figure 8: PR example

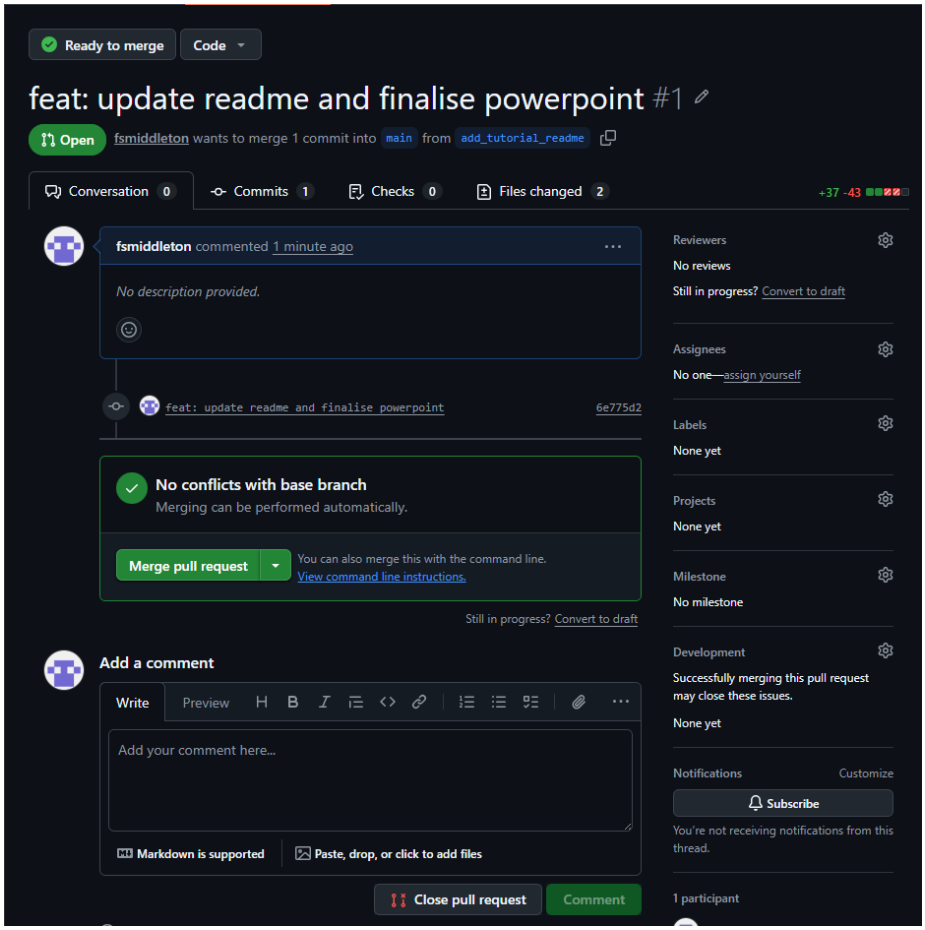


Figure 9: PR ready to merge

- If your repository uses required reviews (via protected branches or rulesets), the PR must receive the specified number of approvals before merging. [docs.github.com], [docs.github.com]
- When checks pass and reviews approve, choose a merge strategy:
  - Merge commit (retains all commits, creates a merge commit).
  - Squash and merge (combines commits into one for a cleaner history).
  - Rebase and merge (replays commits atop the base, producing a linear history).
    GitHub documents when each strategy is appropriate and how it appears in history. [docs.github.com]

Finally, click Merge. After merging, it's common to delete the now-merged topic branch on the server; GitHub can do this automatically if configured. [docs.github.com]

Where each command "fits" in code management

Action	Command(s)	What it means in the broader framework
Clone repository	`git clone https://…`	Creates a full local mirror of the project's history so you can develop and contribute. [docs.github.com]
Create topic branch	`git switch -c feature/...`	Starts an isolated line of development aligned with GitHub Flow: branch → commit → PR → merge. [git-scm.com], [docs.github.com]
Stage/commit	`git add` / `git commit -m "…"`	Records a snapshot of your staged changes with an explanatory message, your atomic unit of history and review. [git-scm.com], [git-scm.com]
Push branch	`git push -u origin feature/...`	Publishes your branch to the remote so collaborators and CI can see it; sets tracking for future sync. [docs.github.com]
Open PR	(on GitHub.com)	A structured request to merge changes from your branch into the base branch, with review and checks. [docs.github.com]
Review/approve	(on GitHub.com)	Enforces quality gates, discussion, code review, and required approvals per repository policy. [docs.github.com]
Merge	(on GitHub.com)	Integrates the approved work; choice of merge strategy affects history shape and traceability. [docs.github.com]

Best practices for PRs (concise checklist)

- Keep PRs small, cohesive, and reviewable; prefer a few focused commits. [docs.github.com]
- Use a clear title and a description that states what and why; link issues
- Provide or update tests and relevant docs in the same PR.
- Follow repository standards: PR templates, CODEOWNERS, protected branches, rulesets. [docs.github.com]
- Write good commit messages (imperative subject, optional wrapped body explaining rationale). [cbea.ms]

Full end-to-end example: recap of the commands

```
# 1. Clone via HTTPS and open in VSCode
git clone https://github.com/ORG/REPO.git
cd REPO
code .

# Create a topic branch
git switch -c feature

# Make your change (edit files)

# Commit feature change
git add .
git commit -m "feat: some message"
# Publish branch
git push -u origin feature

# Open a PR on GitHub.com, request review, and merge after approval
# (GUI steps on GitHub as described above)
```

Troubleshooting and tips

- Can't push? Ensure you have permissions to the organization repo and that your remote (`origin`) points to the correct URL. `git remote -v` and `git remote set-url origin …` help verify/correct remotes. [docs.github.com]
- PR not mergeable? Check for required reviews, status checks, and branch protection rules. [docs.github.com], [docs.github.com]
- Unsure which merge strategy to select? Review GitHub's documentation on merge, squash, and rebase merges and choose the policy your team recommends. [docs.github.com]